# DESIGN AND ANALYSIS OF ALGORITHMS

# CO3 – DIVIDE AND CONQUER TECHNIQUE

# ASSIGNMENT 3 – PAIR WORK

## Q1 – DECOMPOSITION AND ELIMINATION IN ORDERED DATA

### PART A – FINDING MAXIMUM AND MINIMUM

**1. Introduction**

Finding the maximum and minimum elements means identifying the largest and smallest values in an array. The Divide-and-Conquer approach divides the elements into smaller groups, finds the local maximum and minimum, and combines the results.

A tournament-style strategy reduces the number of comparisons compared with finding the maximum and minimum independently.

**2. Algorithm – Tournament Method**

i. Start.

ii. Read the number of elements and the array.

iii. If there is one element, assign it as both minimum and maximum.

iv. Compare the elements in pairs.

v. Store the smaller element as the local minimum and the larger element as the local maximum.

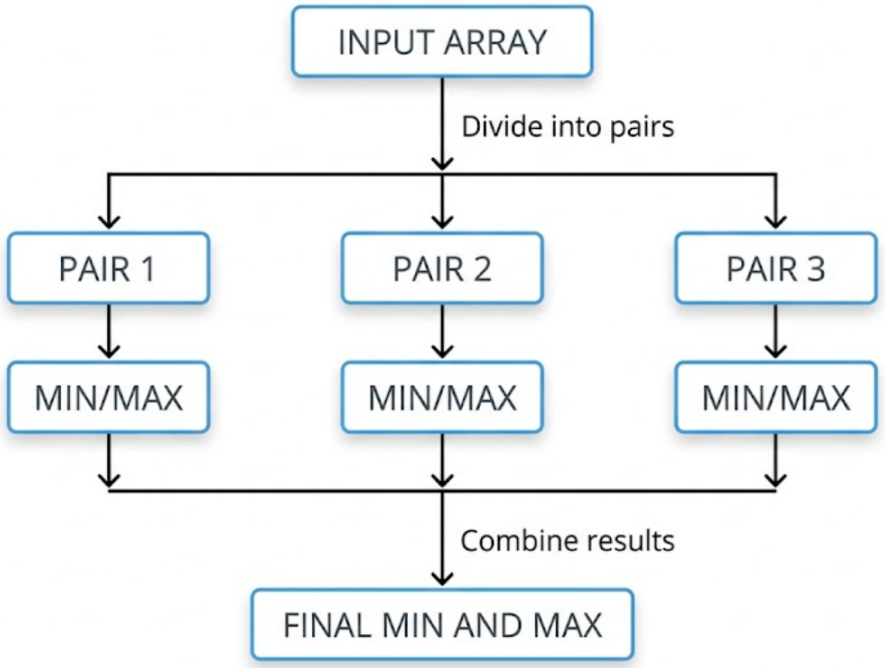
vi. Compare local minimums with the global minimum.

vii. Compare local maximums with the global maximum.

viii. Display the final minimum and maximum.

ix. Stop.

**3. Decomposition Diagram**




Each pair requires one comparison to identify its local minimum and maximum. The local results are then combined to obtain the overall minimum and maximum.

**4. Pseudocode**

```
MINMAX(A, n)
  if n = 1
    return A[0], A[0]
  if n is even
    compare A[0], A[1]
    initialize min and max
    i = 2
  else
    min = max = A[0]
    i = 1
```

while i < n

compare A[i], A[i+1]

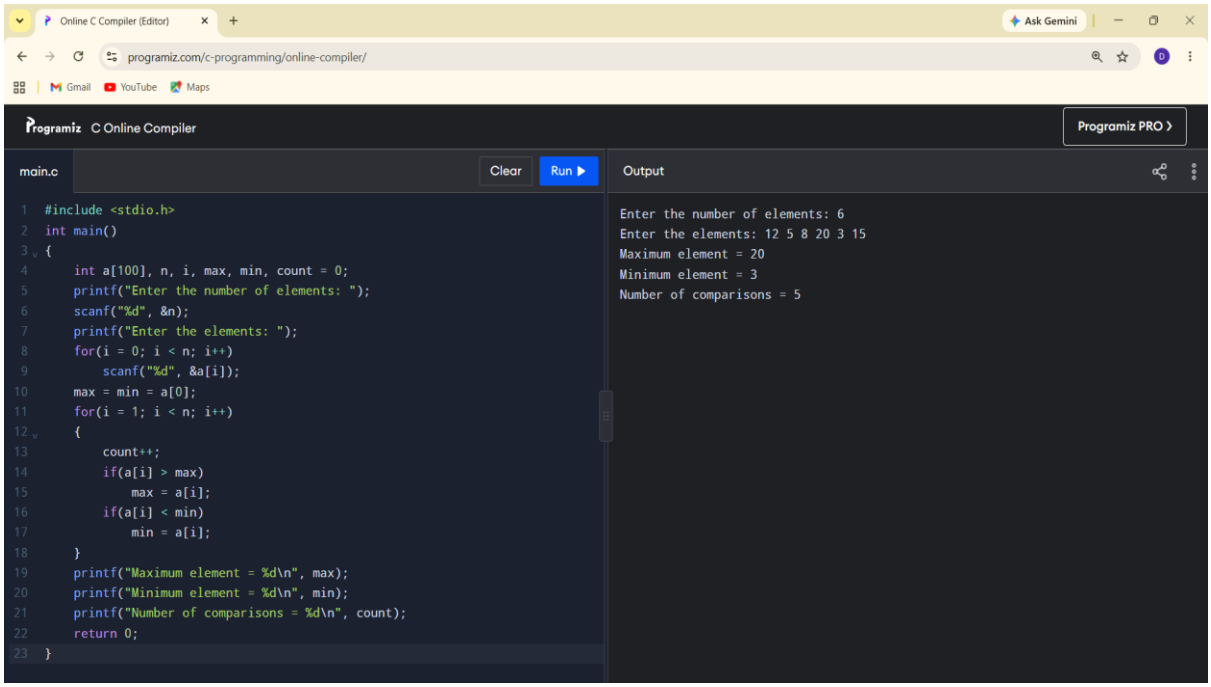
find local min and max

update global min and max

i = i + 2

return min, max

## 5. C Program




## 6. Example

Consider the array:

12, 7, 25, 3, 18, 9, 30, 5

| Pair | Comparison | Local Minimum | Local Maximum |
|---|---|---|---|
| 12, 7 | 12 > 7 | 7 | 12 |
| 25, 3 | 25 > 3 | 3 | 25 |
| 18, 9 | 18 > 9 | 9 | 18 |
| 30, 5 | 30 > 5 | 5 | 30 |

Combining the local results:

- Minimum = min(7, 3, 9, 5) = 3
- Maximum = max(12, 25, 18, 30) = 30

Total comparisons = 10.

**7. Mathematical Analysis**

For an even number of elements:

- First pair requires 1 comparison.
- Each remaining pair requires 3 comparisons: one within the pair, one for the minimum and one for the maximum.

Therefore,

$$C(n) = 1 + 3\left(\frac{n-2}{2}\right)$$

$$\boxed{C(n) = \frac{3n}{2} - 2}$$

For odd $n$, the first element is initially assigned as both minimum and maximum.

$$\boxed{C(n) = \frac{3(n-1)}{2}}$$

The independent method requires:

$$C(n) = (n-1) + (n-1)$$

$$\boxed{C(n) = 2n - 2}$$

For $n = 8$:

- Tournament method = $3(8)/2 - 2 = 10$
- Independent method = $2(8) - 2 = 14$

Thus, the tournament method saves four comparisons.

## 8. Experimental Analysis – Student A and Student B

Student A explains the decomposition, local results and mathematical comparison count. Student B tests the implementation using increasing input sizes and records the comparison counter.

| Input Size | Tournament Comparisons | Independent Comparisons |
|---|---|---|
| 2 | 1 | 2 |
| 4 | 4 | 6 |
| 8 | 10 | 14 |
| 16 | 22 | 30 |
| 32 | 46 | 62 |
| 64 | 94 | 126 |

These are theoretical and expected comparison counts. The program's observed counts should match them when the same pairwise method is implemented and only element comparisons are counted.

Increasing, decreasing and randomly arranged arrays should produce the same comparison count because the algorithm processes every pair in the same manner.

## 9. Complexity Analysis

The algorithm processes every element once.

$$T(n) = \Theta(n)$$

- Time Complexity: $O(n)$
- Auxiliary Space: $O(1)$

## 10. Conclusion

The tournament-based Divide-and-Conquer method finds both the maximum and minimum using fewer comparisons than the independent method. It is efficient for processing large datasets and reduces unnecessary comparisons.

## PART B – BINARY SEARCH

### 1. Introduction

Binary Search is an efficient searching technique used to find an element in a sorted array. It repeatedly divides the search interval into two halves and eliminates the half that cannot contain the target.

### 2. Algorithm

i. Start.

ii. Read the sorted array and target element.

iii. Initialize low = 0 and high = n-1.

iv. Calculate the middle position.

v. If the middle element equals the target, return its position.

vi. If the target is smaller, search the left half.

vii. Otherwise, search the right half.

viii. Repeat until the element is found or low > high.

ix. Display the result and stop.

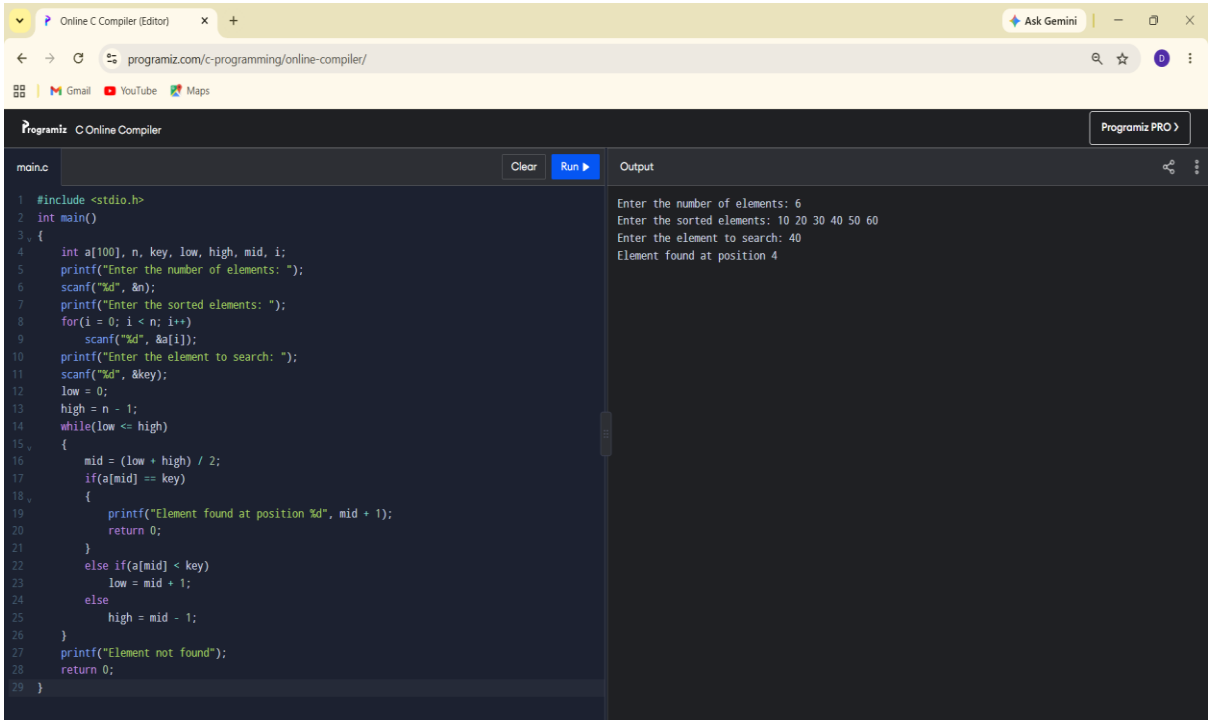
### 3. Decision Rule

| Condition | Action |
|---|---|
| Target = A[mid] | Element found |
| Target < A[mid] | Search left half |
| Target > A[mid] | Search right half |
| low > high | Element not found |

## 4. Pseudocode

```
BINARYSEARCH(A, n, key)
    low = 0
    high = n - 1
    while low <= high
        mid = (low + high) / 2
        if A[mid] = key
            return mid
        else if key < A[mid]
            high = mid - 1
        else
            low = mid + 1
    return -1
```

## 5. C Program



**6. Example**

Sorted array:

10, 20, 30, 40, 50, 60, 70

Target = 60

| Step | Low | High | Mid | Middle Element | Decision |
|---|---|---|---|---|---|
| 1 | 0 | 6 | 3 | 40 | Search right |
| 2 | 4 | 6 | 5 | 60 | Found |

The element 60 is found at position 6.

**7. Elimination of Search Space**

At every iteration, approximately half of the remaining elements are eliminated.

If the target is smaller than the middle element, all elements to the right can be eliminated because the array is sorted. Similarly, if the target is larger, all elements to the left can be eliminated.

This guarantees that the target, if present, remains inside the active search interval.

**8. Successful and Unsuccessful Searches**

- Successful search: The target exists in the array.
- Unsuccessful search: The target does not exist in the array.

For example, searching for 60 in the given array is successful. Searching for 90 is unsuccessful because the search interval eventually becomes empty.

**9. Mathematical Analysis**

At every iteration, the search space becomes half of its previous size.

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots$$

After $k$iterations:

$$\frac{n}{2^k} \leq 1$$
$$2^k \geq n$$
$$k \geq \log_2 n$$

Therefore,

$$\boxed{T(n) = O(\log n)}$$

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$
- Auxiliary Space: $O(1)$for iterative Binary Search.

**10. Experimental Analysis**

Student A tests successful and unsuccessful searches and records the number of midpoint probes. Student B explains the elimination rule and proves why the discarded half cannot contain the target.

| Array Size | Worst-Case Midpoint Probes |
|---|---|
| 8 | 4 |
| 16 | 5 |
| 32 | 6 |
| 64 | 7 |
| 128 | 8 |
| 256 | 9 |

These are theoretical worst-case probe counts for the listed sizes. Actual program counts depend on the target's position and whether the search succeeds.

**11. Conclusion**

Binary Search reduces the search space by half in every iteration. It is significantly faster than linear search for large sorted datasets and has logarithmic time complexity.

## Q2 – QUALITY OF DECOMPOSITION IN SORTING

## PART A – MERGE SORT

### 1. Introduction

Merge Sort is a Divide-and-Conquer sorting algorithm. It divides an array into two halves, recursively sorts each half and merges the sorted halves into one sorted array.

It provides consistent performance even when the input is already sorted or arranged in reverse order.

### 2. Working Principle

i. Divide the array into two halves.

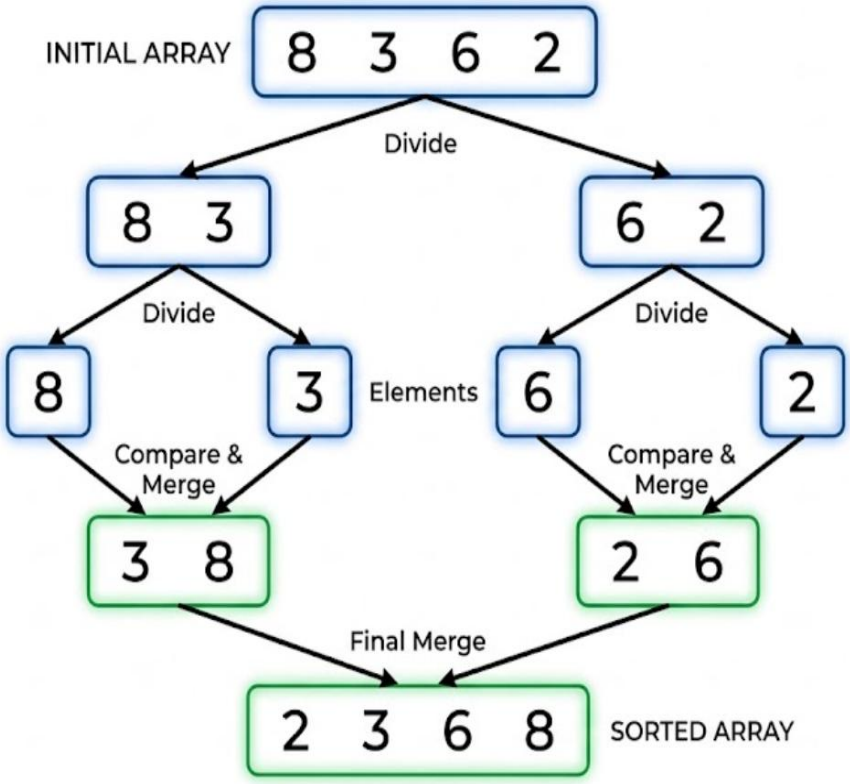
ii. Continue dividing until each subarray contains one element.

iii. Merge two sorted subarrays.

iv. Compare their elements and place the smaller element first.

v. Continue merging until the complete array is sorted.

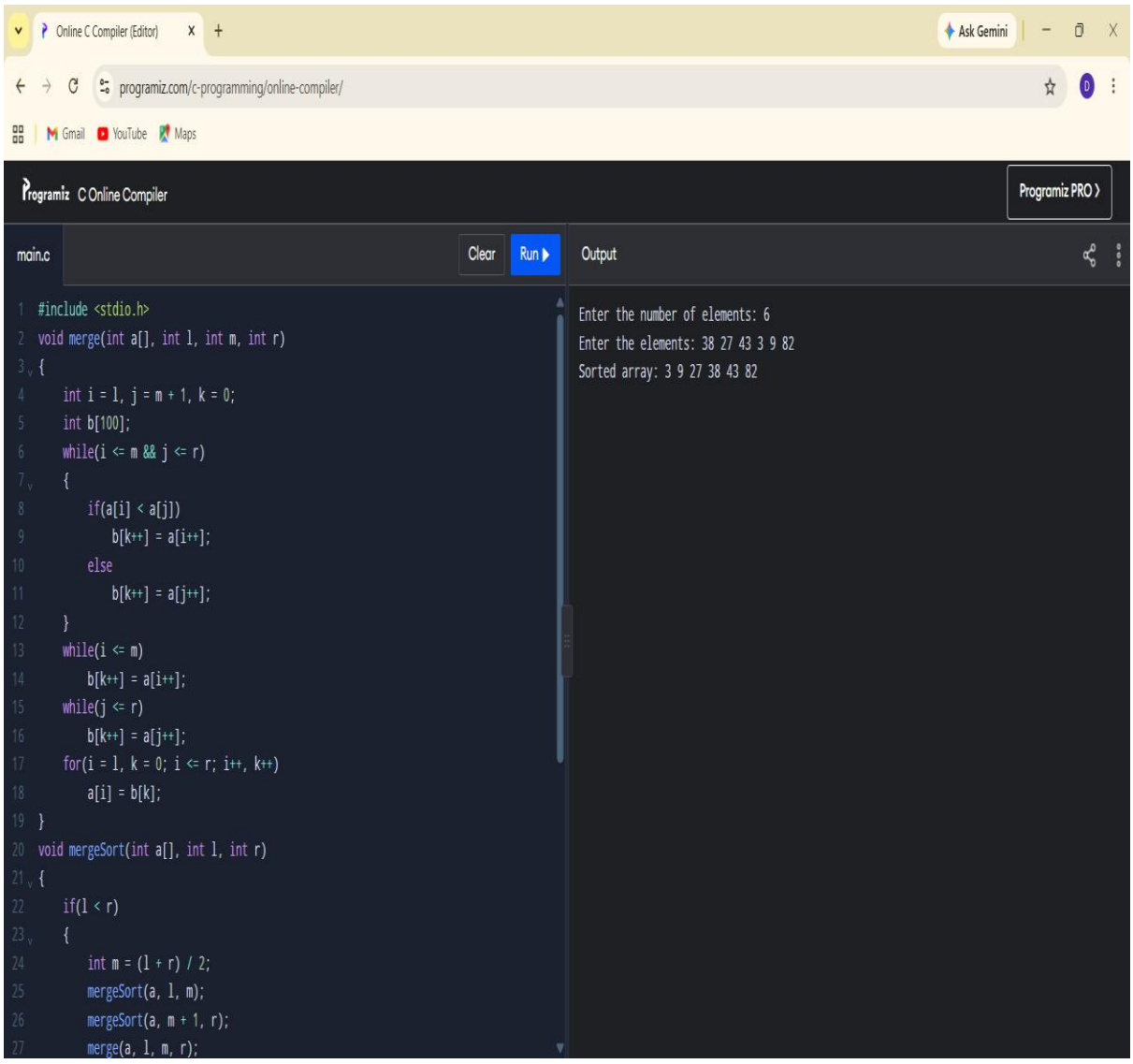
### 3. Decomposition Diagram

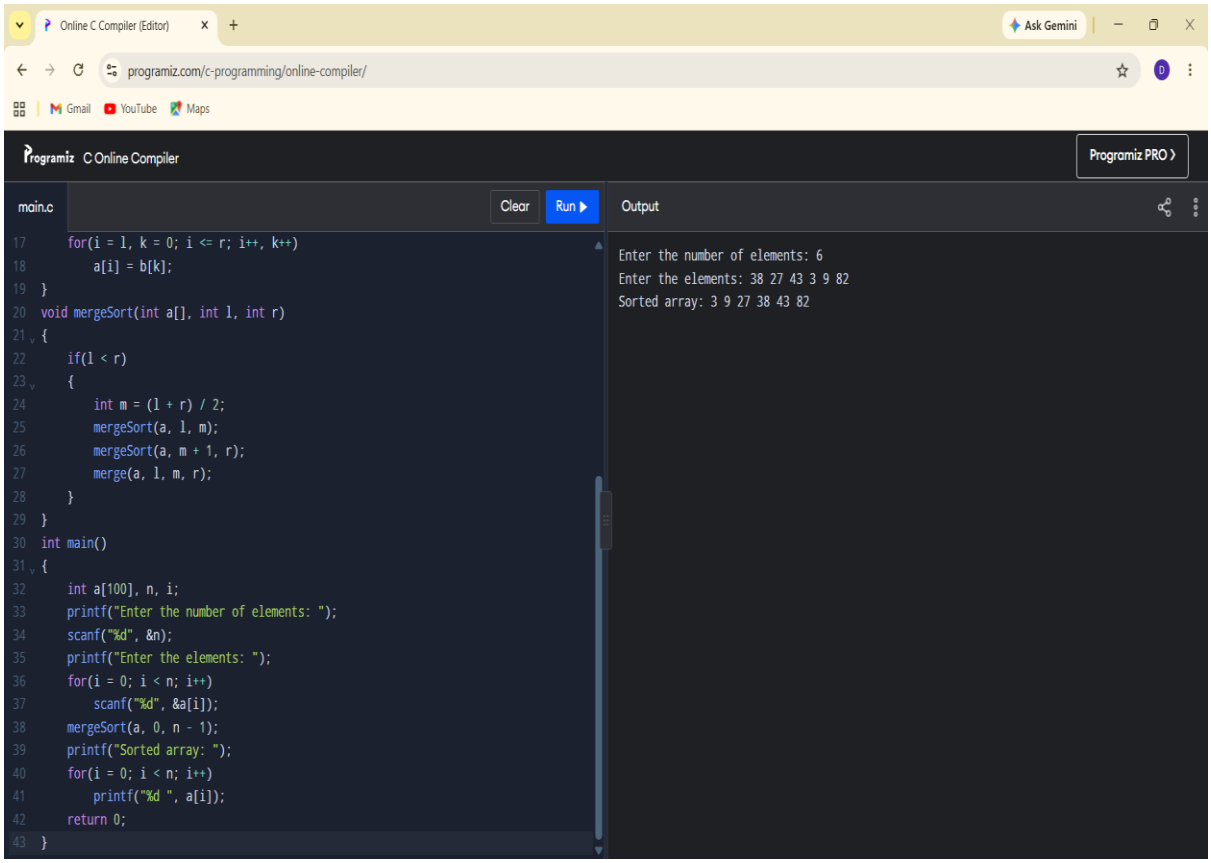


## 4. Pseudocode

```
MERGESORT(A, low, high)
    if low < high
        mid = (low + high) / 2
        MERGESORT(A, low, mid)
        MERGESORT(A, mid+1, high)
        MERGE(A, low, mid, high)
```

## 5. C Program






**6. Example**

Input:

8, 3, 6, 2

Division:

- [8, 3] and [6, 2]
- [8] [3] [6] [2]

Merging:

- [8] and [3] → [3, 8]
- [6] and [2] → [2, 6]
- [3, 8] and [2, 6] → [2, 3, 6, 8]

Output:

2, 3, 6, 8

**7. Mathematical Analysis**

The array is divided into two equal parts and merging requires linear time.

$$T(n) = 2T(n/2) + cn$$

Expanding:

$$T(n) = 4T(n/4) + 2cn$$

After $k$levels:

$$T(n) = nT(1) + cn\log_2 n$$

Therefore,

$$\boxed{T(n) = \Theta(n \log n)}$$

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$
- Auxiliary Space: $O(n)$

**8. Quality of Decomposition**

Merge Sort divides the input into balanced halves.

This produces a recursion tree with approximately $\log_2 n$levels. Every level performs a total of $O(n)$merging work.

Its main advantage is predictable performance. However, it requires additional memory for merging.

**9. Conclusion**

Merge Sort is suitable for large datasets when stable sorting and predictable performance are important. Its balanced decomposition guarantees $O(n \log n)$time complexity.

## PART B – QUICK SORT

### 1. Introduction

Quick Sort is a Divide-and-Conquer sorting algorithm that selects a pivot element and partitions the array into two sections. Elements smaller than the pivot are placed on the left, while larger elements are placed on the right.

The same process is recursively applied to both sections.

### 2. Working Principle

i. Select a pivot element.

ii. Partition the array around the pivot.

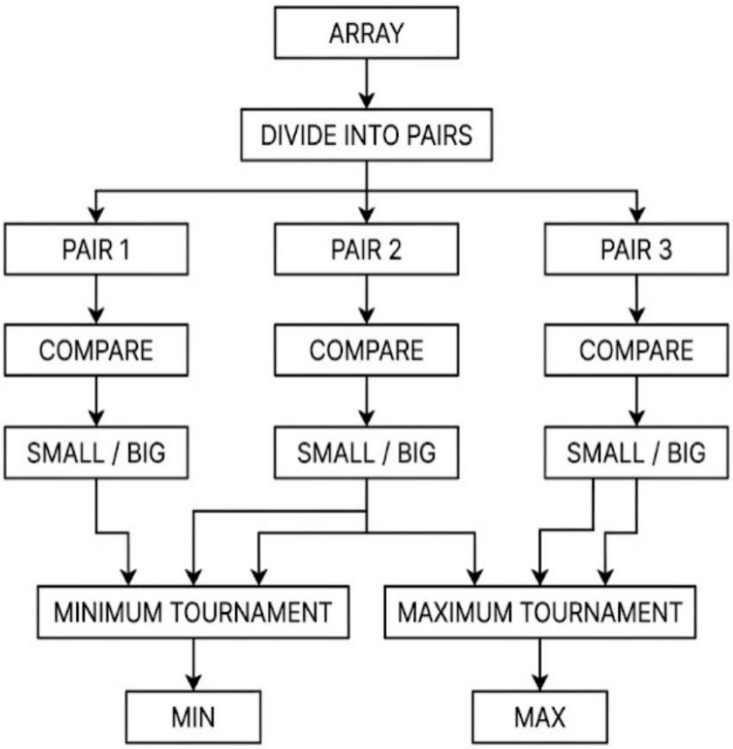
iii. Place smaller elements before the pivot.

iv. Place larger elements after the pivot.

v. Recursively sort the left and right partitions.

vi. Stop when a partition contains zero or one element.

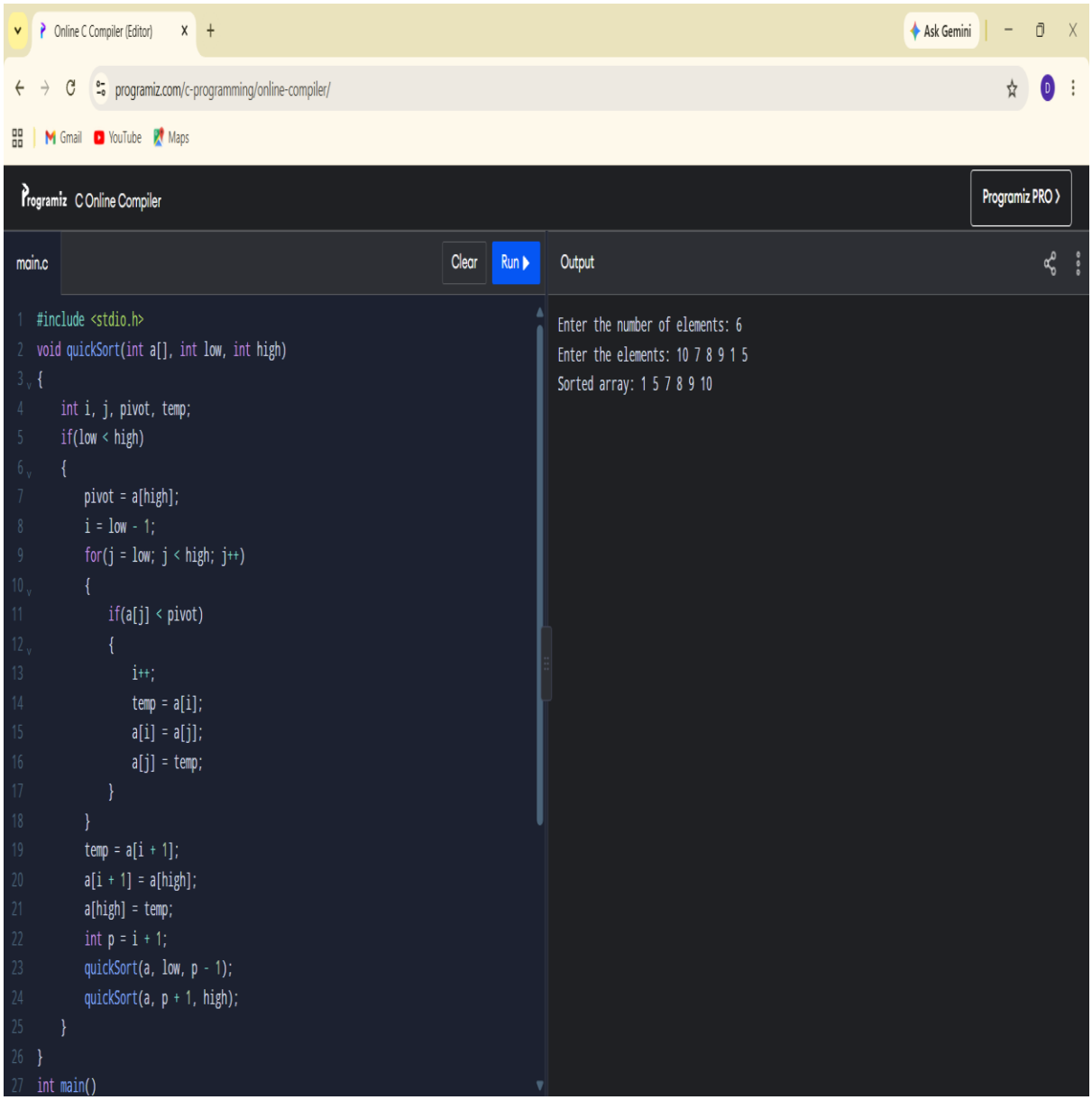
### 3. Decomposition Diagram

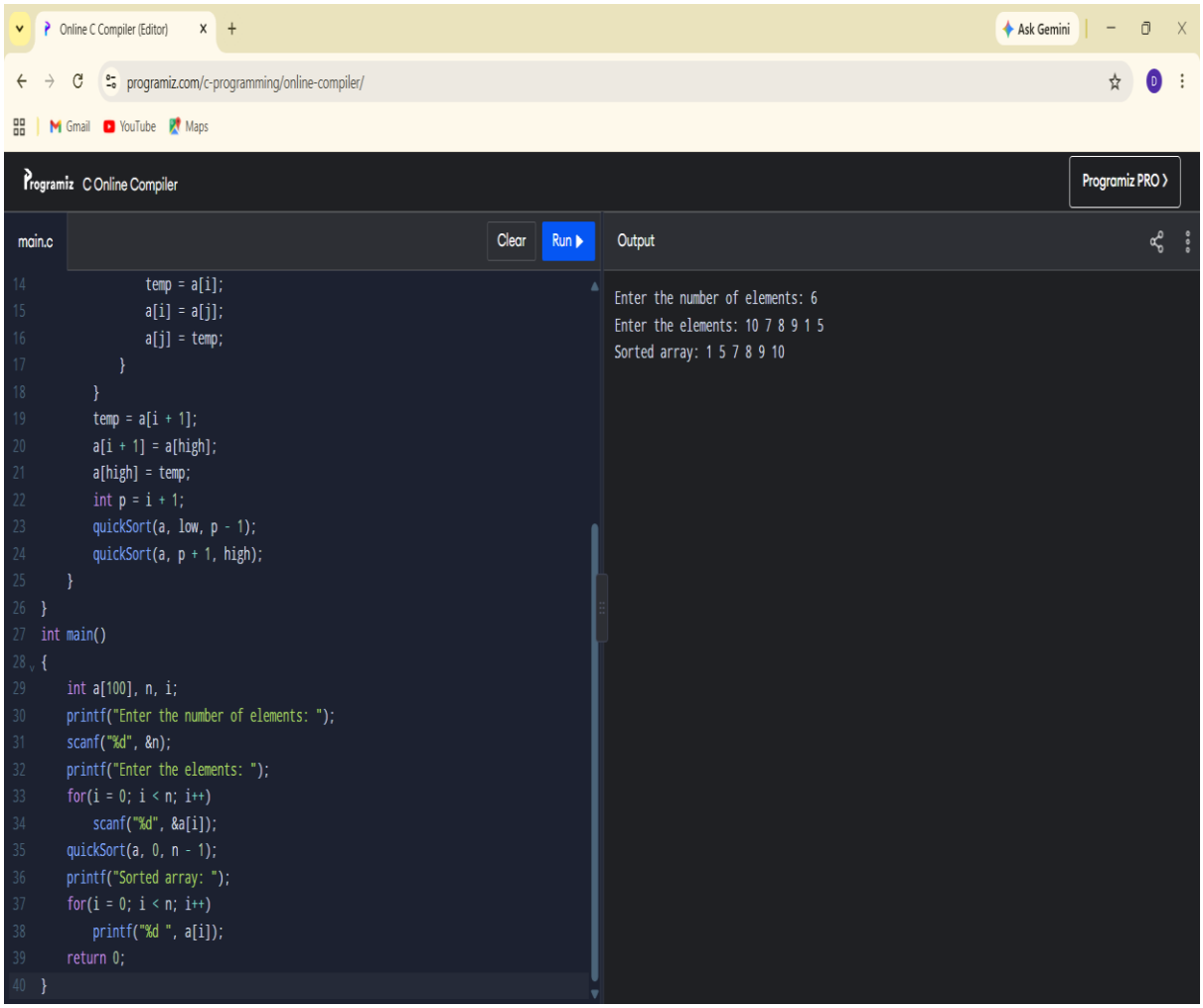


**4. Pseudocode**

```
QUICKSORT(A, low, high)
    if low < high
        p = PARTITION(A, low, high)
        QUICKSORT(A, low, p-1)
        QUICKSORT(A, p+1, high)
```

**5. C Program**






**6. Example**

Input:

6, 3, 8, 2, 7

Choose the last element, 7, as pivot.

After partitioning:

6, 3, 2 | 7 | 8

The left partition is sorted into 2, 3, 6, and the right partition contains 8.

Final output:

2, 3, 6, 7, 8

**7. Mathematical Analysis**

**Best and Average Case:**

When the pivot divides the array into nearly equal parts:

$$T(n) = 2T(n/2) + cn$$

$$\boxed{T(n) = \Theta(n \log n)}$$

**Worst Case:**

When the pivot is always the smallest or largest element:

$$T(n) = T(n-1) + cn$$

Expanding:

$$T(n) = c(n + n - 1 + \cdots + 1)$$

$$T(n) = \frac{cn(n+1)}{2}$$

$$\boxed{T(n) = \Theta(n^2)}$$

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$
- Auxiliary Space: $O(\log n)$on average for recursion; $O(n)$in the worst case.

**8. Comparison of Merge Sort and Quick Sort**

| Feature | Merge Sort | Quick Sort |
|---|---|---|
| Technique | Divide and merge | Pivot and partition |
| Best Case | $O(n \log n)$ | $O(n \log n)$ |
| Average Case | $O(n \log n)$ | $O(n \log n)$ |
| Worst Case | $O(n \log n)$ | $O(n^2)$ |
| Extra Space | $O(n)$ | Usually $O(\log n)$ |
| Stability | Stable | Usually unstable |
| Main Advantage | Predictable performance | Efficient in-place sorting |

**9. Experimental Evaluation**

Test both algorithms using:

- Randomly arranged input.
- Already sorted input.
- Reverse-sorted input.
- Increasing input sizes.

Record the execution time and comparison count separately. For Quick Sort, using the last element as pivot can produce poor performance on already sorted input. Merge Sort maintains its $O(n \log n)$time complexity across these arrangements.

Do not treat theoretical counts as measured timings. Record actual execution times by running both programs under the same conditions.

**10. Conclusion**

Merge Sort provides guaranteed $O(n \log n)$performance but requires additional memory. Quick Sort is often efficient in practice and uses less auxiliary memory, but its performance depends on pivot selection. The suitable algorithm depends on memory availability and performance requirements.

## Q3 – KARATSUBA MULTIPLICATION

**1. Introduction**

Karatsuba multiplication is a Divide-and-Conquer algorithm used to multiply large integers faster than the traditional multiplication method. It reduces four smaller multiplications into three.

**2. Mathematical Principle**

Let two numbers be divided into two parts:

$$x = a10^m + b$$
$$y = c10^m + d$$

Normal multiplication requires:

$$xy = ac10^{2m} + (ad + bc)10^{m} + bd$$

Karatsuba calculates:

$$P = ac,\ Q = bd$$
$$R = (a + b)(c + d)$$

Then:

$$ad + bc = R - P - Q$$

Therefore,

$$\boxed{xy = P10^{2m} + (R - P - Q)10^{m} + Q}$$

Only three recursive multiplications are required.

**3. Algorithm**

i. Divide both numbers into two halves.

ii. Calculate $P = ac$.

iii. Calculate $Q = bd$.

iv. Calculate $R = (a + b)(c + d)$.

v. Find the middle term $R - P - Q$.

vi. Combine the three results.

**4. Pseudocode**

```
KARATSUBA(x, y)
  if x or y is single digit
    return x * y
  divide x into a, b
  divide y into c, d
  p = KARATSUBA(a, c)
```

q = KARATSUBA(b, d)

r = KARATSUBA(a+b, c+d)

return p*10^(2m) + (r-p-q)*10^m + q

**5. Example**

Multiply 12 and 34.

$$a = 1, b = 2, c = 3, d = 4$$
$$P = 1 \times 3 = 3$$
$$Q = 2 \times 4 = 8$$
$$R = (1 + 2)(3 + 4) = 3 \times 7 = 21$$
$$R - P - Q = 21 - 3 - 8 = 10$$

Therefore,

$$12 \times 34 = 3(100) + 10(10) + 8$$
$$\boxed{408}$$

**6. Complexity Analysis**

Karatsuba performs three recursive multiplications on numbers of half the original size.

$$T(n) = 3T(n/2) + O(n)$$

Using the Master Theorem:

$$T(n) = \Theta\left(n^{\log_2 3}\right)$$
$$\boxed{T(n) \approx O(n^{1.585})}$$

Traditional multiplication requires $O(n^2)$time.

**7. Conclusion**

Karatsuba multiplication improves large-number multiplication by reducing four recursive multiplications to three. It is useful in applications involving very large integers.

## Q4 – STRASSEN MATRIX MULTIPLICATION

### 1. Introduction

Strassen's algorithm is a Divide-and-Conquer method for multiplying two square matrices. It improves the conventional matrix multiplication approach by reducing eight recursive multiplications to seven.

### 2. Mathematical Principle

Divide matrices A and B into four submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Calculate seven products:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_2 = (A_{21} + A_{22})B_{11}$$
$$M_3 = A_{11}(B_{12} - B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{12})B_{22}$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

The resulting submatrices are:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{22} = M_1 - M_2 + M_3 + M_6$$

### 3. Algorithm

i. Divide both matrices into four submatrices.
ii. Calculate the seven intermediate products.
iii. Combine them using the four result equations.
iv. Repeat recursively for larger matrices.
v. Return the final product matrix.

**4. Pseudocode**

STRASSEN(A, B)

if size = 1

return A * B

divide A and B into four blocks

calculate M1 to M7

calculate C11, C12, C21, C22

combine C blocks

return C

**5. Example**

Consider:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$M_1 = (1 + 4)(5 + 8) = 65$$

$$M_2 = (3 + 4)(5) = 35$$

$$M_3 = 1(6 - 8) = -2$$

$$M_4 = 4(7 - 5) = 8$$

$$M_5 = (1 + 2)(8) = 24$$

$$M_6 = (3 - 1)(5 + 6) = 22$$

$$M_7 = (2 - 4)(7 + 8) = -30$$

Combining:

$$C_{11} = 65 + 8 - 24 - 30 = 19$$

$$C_{12} = -2 + 24 = 22$$

$$C_{21} = 35 + 8 = 43$$

$$C_{22} = 65 - 35 - 2 + 22 = 50$$

Final matrix:

$$\boxed{C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}}$$

**6. Complexity Analysis**

$$T(n) = 7T(n/2) + O(n^2)$$

Using the Master Theorem:

$$\boxed{T(n) = O(n^{\log_2 7})}$$

$$\boxed{T(n) \approx O(n^{2.807})}$$

Conventional matrix multiplication requires $O(n^3)$time.

**7. Conclusion**

Strassen's algorithm reduces the number of recursive matrix multiplications from eight to seven.

It is useful for large matrices, although its additional additions, subtractions and implementation overhead can make it less suitable for small matrices.

## Q5 – CLOSEST PAIR OF POINTS USING DIVIDE AND CONQUER

**1. Introduction**

The Closest Pair of Points problem finds two points with the minimum Euclidean distance in a given set of points.

The Divide-and-Conquer approach divides the points into two halves, finds the closest pair in each half and checks points near the dividing line.

**2. Mathematical Formula**

For two points:

$$P_1 = (x_1, y_1),\ P_2 = (x_2, y_2)$$

The Euclidean distance is:

$$\boxed{d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}$$

**3. Algorithm**

i. Sort points according to their x-coordinates.

ii. Divide the points into two halves.

iii. Find the closest pair in the left half.

iv. Find the closest pair in the right half.

v. Select the smaller distance.

vi. Check points in the middle strip that may form a closer pair.

vii. Return the minimum distance.

**4. Pseudocode**

```
CLOSESTPAIR(P)
    if number of points <= 3
        find minimum distance directly
    divide P into left and right halves
    d1 = CLOSESTPAIR(left)
    d2 = CLOSESTPAIR(right)
    d = min(d1, d2)
    check middle strip for smaller distance
    return minimum distance
```

**5. Example**

Consider four points:

$$P_1 = (1, 1), P_2 = (2, 2)$$
$$P_3 = (5, 5), P_4 = (6, 5)$$

Calculate the distances:

$$d(P_1, P_2) = \sqrt{(2-1)^2 + (2-1)^2} = \sqrt{2}$$
$$d(P_3, P_4) = \sqrt{(6-5)^2 + (5-5)^2} = 1$$

The minimum distance is:

$$\boxed{d_{\min} = 1}$$

The closest pair is $P_3$and $P_4$.

**6. Complexity Analysis**

The points are divided into two halves. The two recursive calls take $2T(n/2)$, while checking the middle strip takes linear time when the points are maintained in the required sorted order.

$$T(n) = 2T(n/2) + O(n)$$

Therefore,

$$\boxed{T(n) = O(n \log n)}$$

- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$for auxiliary storage in a standard implementation.

**7. Conclusion**

The Divide-and-Conquer method improves the Closest Pair problem from the brute-force $O(n^2)$approach to $O(n \log n)$. It is useful in geometry, computer graphics and spatial data processing.

## Q6 – ADVANCED DIVIDE-AND-CONQUER MINI-INVESTIGATIONS

## PART A – MEDIAN OF MEDIANS (STUDENT A)

**1. Introduction**

Median of Medians is a selection algorithm used to find the kth smallest element in an unsorted array. It selects a reliable pivot to ensure that the partitions remain sufficiently balanced.

**2. Working Principle**

i. Divide the array into groups of five elements.

ii. Find the median of each group.

iii. Find the median of those medians.

iv. Use it as the pivot.

v. Partition the array around the pivot.

vi. Recursively search the required partition.

**3. Pseudocode**

```
SELECT(A, k)
   divide A into groups of five
   find each group's median
   pivot = median of medians
   partition A around pivot
   if pivot position = k
      return pivot
   if k is smaller
      search left partition
   else
      search right partition
```

**4. Example**

Consider:

12, 3, 5, 7, 4, 19, 26, 8, 1

Groups:

- Group 1: 12, 3, 5, 7, 4 → median = 5
- Group 2: 19, 26, 8, 1 → median = 8 (after sorting: 1, 8, 19, 26, choose the upper middle for this even-sized group)

The pivot is selected from the group medians. Partitioning around this pivot helps reduce the search space.

**5. Complexity Analysis**

The recurrence is:

$$T(n) \leq T(n/5) + T(7n/10) + O(n)$$

The algorithm guarantees:

$$\boxed{T(n) = O(n)}$$

It provides worst-case linear-time selection.

**6. Conclusion**

Median of Medians guarantees a good pivot and avoids the poor worst-case behaviour of ordinary Quick Select. It is useful when guaranteed selection performance is required.

## PART B – MEET-IN-THE-MIDDLE SUBSET SUM (STUDENT B)

**1. Introduction**

Meet-in-the-Middle is an algorithm used to solve the Subset Sum problem. It divides the input array into two halves, calculates possible subset sums for each half and searches for a combination equal to the target.

**2. Working Principle**

i. Divide the array into two halves.

ii. Generate all subset sums of the first half.

iii. Generate all subset sums of the second half.

iv. Check whether two sums add up to the target.

v. If a matching pair exists, the subset is found.

**3. Pseudocode**

```
SUBSETSUM(A, target)
    divide A into two halves
    generate sums S1 and S2
```

```
    for each x in S1
        if target - x exists in S2
            return TRUE
    return FALSE
```

**4. Worked Example**

Array:

3, 5, 7, 10

Target = 15

Divide into:

- First half: 3, 5
- Second half: 7, 10

Subset sums:

$$S_1 = \{0, 3, 5, 8\}$$
$$S_2 = \{0, 7, 10, 17\}$$

Choose:

$$5 + 10 = 15$$

Therefore, the required subset is:

$$\boxed{\{5, 10\}}$$

**5. Complexity Analysis**

For $n$elements, each half contains approximately $n/2$elements.

Number of subset sums per half:

$$2^{n/2}$$

Generating and sorting the sums takes approximately:

$$O\left(2^{n/2} \log 2^{n/2}\right)$$

Therefore,

$$\boxed{T(n) = O\left(n2^{n/2}\right)}$$

With suitable sorting and binary search, the search can be performed efficiently. The space requirement is:

$$\boxed{O\left(2^{n/2}\right)}$$

**6. Conclusion**

Meet-in-the-Middle reduces the search effort compared with checking all $2^n$ subsets directly. It is useful for moderate-sized Subset Sum problems where a complete search is too expensive.

## OVERALL CONCLUSION

Divide-and-Conquer solves complex problems by dividing them into smaller subproblems, solving them independently and combining their results. Algorithms such as Merge Sort, Quick Sort, Karatsuba multiplication, Strassen multiplication and Closest Pair demonstrate how decomposition can improve efficiency. Advanced methods such as Median of Medians and Meet-in-the-Middle further improve selection and subset-search problems. Meet-in-the-Middle is especially useful when the input size is too large for checking every possible subset directly. By dividing the elements into two smaller groups, it reduces the number of combinations that need to be examined. It stores the subset sums and uses sorting and binary search to find the required combination.

| Q.No. | Question |
|---|---|
| Q1 (A) | Finding Maximum and Minimum |
| Q1 (B) | Binary Search |
| Q2 (A) | Merge Sort |
| Q2 (B) | Quick Sort |
| Q3 | Karatsuba Multiplication |
| Q4 | Strassen Matrix Multiplication |
| Q5 | Closest Pair Using Divide and Conquer |
| Q6 (A) | Median of Medians |
| Q6 (B) | Meet-in-the-Middle Subset Sum |